

PRACTICAL : WEEK-5

Task - 1:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

CODE:

```
on 1000 0107
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main()
{
    int fd[2];
    pid_t pid;
    int data;

    struct timespec start, end;

    pipe(fd);

    pid = fork();

    if (pid == 0)
    {
        // Child - Consumer

        close(fd[1]);

        printf("Child (Consumer): Receiving data...\n");

        while (read(fd[0], &data, sizeof(data)) > 0)
        {
            printf("Consumed: %d\n", data);
        }

        close(fd[0]);

        return 0;
    }
    else
    {
        // Parent - Producer

        close(fd[0]);

        clock_gettime(CLOCK_MONOTONIC, &start);

        printf("Parent (Producer): Sending data...\n");

        for (int i = 1; i <= 100; i++)
        {
            write(fd[1], &i, sizeof(i));
        }

        close(fd[1]);

        wait(NULL);
        clock_gettime(CLOCK_MONOTONIC, &end);

        double time_taken =
            (end.tv_sec - start.tv_sec) +
            (end.tv_nsec - start.tv_nsec) / 1e9;

        printf("\nCommunication completed.\n");
        printf("Communication time: %f seconds\n", time_taken);

        return 0;
    }
}
```

OUTPUTS:

```
b.deepak@BDeepaks-MacBook-Air ~ % gcc producer-consumer.c -o producer-consumer
b.deepak@BDeepaks-MacBook-Air ~ % ./producer-consumer
Parent (Producer): Sending data...
Child (Consumer): Receiving data...
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
Consumed: 6
Consumed: 7
Consumed: 8
Consumed: 9
Consumed: 10
Consumed: 11
Consumed: 12
Consumed: 13
Consumed: 14
Consumed: 15
Consumed: 16
Consumed: 17
Consumed: 18
Consumed: 19
Consumed: 20
Consumed: 21
Consumed: 22
Consumed: 23
Consumed: 24
Consumed: 25
Consumed: 26
Consumed: 27
Consumed: 28
Consumed: 29
Consumed: 30
Consumed: 31
Consumed: 32
Consumed: 33
Consumed: 34
Consumed: 35
Consumed: 36
Consumed: 37
Consumed: 38
Consumed: 39
Consumed: 40
Consumed: 41
Consumed: 42
Consumed: 43
Consumed: 44
Consumed: 45
Consumed: 46
Consumed: 47
Consumed: 48
Consumed: 49
Consumed: 50
Consumed: 51
Consumed: 52
Consumed: 53
Consumed: 54
Consumed: 55
Consumed: 56
Consumed: 57
Consumed: 58
Consumed: 59
Consumed: 60
Consumed: 61
```

```
Consumed: 64
Consumed: 65
Consumed: 66
Consumed: 67
Consumed: 68
Consumed: 69
Consumed: 70
Consumed: 71
Consumed: 72
Consumed: 73
Consumed: 74
Consumed: 75
Consumed: 76
Consumed: 77
Consumed: 78
Consumed: 79
Consumed: 80
Consumed: 81
Consumed: 82
Consumed: 83
Consumed: 84
Consumed: 85
Consumed: 86
Consumed: 87
Consumed: 88
Consumed: 89
Consumed: 90
Consumed: 91
Consumed: 92
Consumed: 93
Consumed: 94
Consumed: 95
Consumed: 96
Consumed: 97
Consumed: 98
Consumed: 99
Consumed: 100
```

OBSERVATION:

The parent process successfully acted as the producer and generated data, which was sent to the child process through an anonymous pipe. The child process acted as the consumer and successfully received and displayed the data using the read end of the pipe. The communication between the parent and child processes was successfully established using `pipe()` and `fork()`. The time taken for the producer-consumer communication was measured using `clock_gettime()`, allowing the communication efficiency to be observed. The experiment demonstrates that anonymous pipes can be effectively used for communication between related processes.

Task - 2:

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    pipe(fd);

    pid1 = fork();

    if (pid1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);

        return 0;
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);

        return 0;
    }

    close(fd[0]);
    close(fd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

OUTPUT:

```
b.deepak@BDeepaks-MacBook-Air ~ % nano pipeline.c
b.deepak@BDeepaks-MacBook-Air ~ % gcc pipeline.c -o pipeline
b.deepak@BDeepaks-MacBook-Air ~ % ./pipeline
drwx-----@    5 b.deepak  staff    160   2 Mar  2026 Applications
-rwxr-xr-x      1 b.deepak  staff  33720   5 Aug 09:15 copyfile
-rw-r--r--      1 b.deepak  staff    577   5 Aug 09:21 copyfile.c
drwx-----+   41 b.deepak  staff   1312  15 Sep 20:00 Documents
-rw-r--r--      1 b.deepak  staff   2798   3 Dec  2025 Exp:7 DDCA.circ
-rw-r--r--      1 b.deepak  staff    749   5 Aug 09:11 file1.c
-rw-r--r--      1 b.deepak  staff     80  25 Jul 09:12 first.c
drwx-----+    5 b.deepak  staff    160  21 Jul  2025 Music
-rw-r--r--      1 b.deepak  staff    763   3 Aug 08:40 os.c
drwx-----+    7 b.deepak  staff    224  20 Apr 20:26 Pictures
-rw-r--r--      1 b.deepak  staff    627  15 Sep 20:01 pipeline.c
-rwxr-xr-x      1 b.deepak  staff  33672   8 Aug 09:24 process
-rw-r--r--      1 b.deepak  staff    826   8 Aug 09:23 process.c
-rwxr-xr-x      1 b.deepak  staff  33768   8 Aug 09:27 process1
-rw-r--r--      1 b.deepak  staff   1164   8 Aug 09:26 process1.c
-rwxr-xr-x      1 b.deepak  staff  33872  15 Sep 12:34 producer-consumer
-rw-r--r--      1 b.deepak  staff   1128  15 Sep 12:35 producer-consumer.c
drwxr-xr-x+     5 b.deepak  staff    160  10 Apr  2025 Public
drwxr-xr-x@     5 b.deepak  staff    160   4 Jul 14:22 PyCharmMiscProject
-rw-r--r--      1 b.deepak  staff   3000  19 Aug 20:23 S4task1.c
-rw-r--r--      1 b.deepak  staff     0    5 Aug 08:53 skil.c
-rw-r--r--      1 b.deepak  staff    352   5 Aug 08:54 skill.c
-rw-r--r--      1 b.deepak  staff    577   5 Aug 08:43 skill1.c
-rw-r--r--      1 b.deepak  staff   1215  11 Aug 12:26 skill4.c
-rw-r--r--      1 b.deepak  staff    696   4 Aug 08:46 task1.c
-rw-----      1 b.deepak  staff   1043   5 Aug 08:50 task1.c.save
drwxr-xr-x      9 b.deepak  staff    288   5 Jul 18:24 vs code
-rw-r--r--      1 b.deepak  staff   1166  11 Aug 12:30 week4S.c
b.deepak@BDeepaks-MacBook-Air ~ %
```

OBSERVATION:

The program successfully implemented the equivalent of the shell command `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls. The first child process executed `ls -l` and redirected its output to the pipe, while the second child process received the pipe output as its input and executed `grep ".c"`. The required `.c` files were successfully displayed as the output. The parent process used `waitpid()` to synchronize with both child processes before terminating. Thus, inter-process communication and command piping were successfully demonstrated.